MoA Prediction — Multi-Model Ensemble

Kaggle competition: Mechanisms of Action (MoA) Prediction

Multi-label classification of 206 biological mechanisms of action from gene-expression and cell-viability profiles.

Workflow

This notebook is organised into the following numbered steps. Section headers and in-code comments below are tagged with the matching step number, so any block of code can be traced back to its place in the pipeline.

1. **Setup** — imports and global configuration
2. **Data loading and cleaning**
3. **Feature engineering** — row statistics, categorical encoding
4. **Model architectures** — MLP, TabularResNet, FT-Transformer
5. **Evaluation metric** — mean multi-label binary log loss
6. **Loss variants** — focal loss and label smoothing
7. **Per-fold training routine** — single model, single fold, single seed
8. **Per-fold preprocessing pipelines** — RankGauss(+PCA) and RankGauss-only variants
9. **Cross-validation — neural network ensemble** (MLP, ResNet, FT-Transformer), multi-seed bagging, parallelised across GPUs
10. **Third model — CatBoost** on raw (non-PCA) features
11. **Ensemble blending** — nested-CV weight search over the four base models
12. **Experiment logging**
13. **Submission building and sanity checks**
14. **Main pipeline** — runs steps 2–13 end to end and writes `submission.csv`

Note: cell outputs and execution numbers in this delivered notebook were reset before upload. Run *Restart & Run All* in an environment with GPU access and the competition data available (see `software/README.md`) to reproduce a clean, sequentially-numbered, fully executed version.

1. Setup: imports and configuration

Installation of `iteractive-stratification` library with `pip`, as it is not included in `scikit-learn` standard distribution. This is necessary for multi-label stratified cross-validation, which ensures that the proportion of positive classes for all 206 targets is balanced in each fold.

```
!pip install iterative-stratification
```

Import all necessary libraries (PyTorch, Pandas, Scikit-Learn, Joblib) and set the global experiment constants.

```
import os
import json
import time
import warnings
import math

# Set before importing PyTorch to reduce GPU memory fragmentation
# when multiple parallel processes allocate tensors.
os.environ.setdefault("PYTORCH_CUDA_ALLOC_CONF", "expandable_segments:True")
```

```
import numpy as np
import pandas as pd
import joblib
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from sklearn.decomposition import PCA
from sklearn.preprocessing import QuantileTransformer
from sklearn.feature_selection import VarianceThreshold
from sklearn.exceptions import DataConversionWarning
from iterstrat.ml_stratifiers import MultilabelStratifiedKFold
from scipy.optimize import minimize
from joblib import Parallel, delayed
from joblib.externals.loky import get_reusable_executor

warnings.filterwarnings("ignore", category=DataConversionWarning)
warnings.filterwarnings("ignore", category=UserWarning)
warnings.filterwarnings("ignore", category=FutureWarning)

DATA_DIR = "/kaggle/input/competitions/lish-moa"
WORK_DIR = "/kaggle/working"
N_FOLDS = 5
RANDOM_STATE = 42
N_QUANTILES = 100
PCA_G_COMPONENTS = 50       # for 'g-...' components
PCA_C_COMPONENTS = 15       # for 'c-...' components
BATCH_SIZE = 256
EPOCHS = 35
LR = 1e-3
WEIGHT_DECAY = 1e-5
CLIP_EPS = 1e-15
CTL_LABEL = "ctl_vehicle"

os.makedirs(WORK_DIR, exist_ok=True)

N_GPUS = torch.cuda.device_count()
DEVICE = torch.device("cuda" if N_GPUS > 0 else "cpu")
USE_AMP = N_GPUS > 0                    # mixed precision (fp16) only if a GPU is available

SEEDS = [42, 202, 2024]                 # multi-seed bagging
EARLY_STOPPING_PATIENCE = 8             # epochs without improvement before stopping
VARIANCE_THRESHOLD = 0.0                # 0.0 = removes only constant columns
```

```
USE_FOCAL_LOSS = False                    # True to use focal loss instead of BCE
FOCAL_ALPHA = 0.25
FOCAL_GAMMA = 2.0
LABEL_SMOOTHING = 0.001                   # 0.0 to disable
LOG_DIR = f"{WORK_DIR}/experiment_logs"

# CatBoost configs
N_ESTIMATORS_CATBOOST = 250
CATBOOST_LEARNING_RATE = 0.08
CATBOOST_DEPTH = 5

# Parallel jobs for the neural networks: 2 per GPU (overlaps compute and
# launch overhead), so both T4s stay busy for the whole CV instead of
# running fold-by-fold, seed-by-seed, model-by-model in strict sequence.
# mlp, resnet and ft_transformer all draw from this one pool (3 models x 3
# seeds x 5 folds = 45 jobs).
#
# CatBoost has its own separate, sequential
# per-fold CV, on features without PCA.
NN_JOBS_PER_GPU = 2
N_JOBS_NN = max(1, N_GPUS * NN_JOBS_PER_GPU)

# The FT-Transformer's batch dimension is highly reduced compared to MLP/ResNet ones
# in order to avoid Out Of Memory errors: self-attention has quadratic computational
# cost O(L^2) relative to the number of features (~870 tokens per line)
BATCH_SIZE_BY_MODEL = {"mlp": BATCH_SIZE, "resnet": BATCH_SIZE, "ft_transformer": 64}
INFERENCE_BATCH_SIZE_BY_MODEL = {"mlp": 1024, "resnet": 1024, "ft_transformer": 128}

print(f"GPUs detected: {N_GPUS} | AMP: {USE_AMP} | parallel NN jobs: {N_JOBS_NN}")

torch.backends.cudnn.benchmark = True
```

2. Data loading and cleaning

Definition of `load_raw_data` and `clean_and_align` functions in order to import CSV files, remove null values or duplicates and align datasets.

```
def load_raw_data(data_dir):
    features = pd.read_csv(f"{data_dir}/train_features.csv")
    targets = pd.read_csv(f"{data_dir}/train_targets_scored.csv")
    test = pd.read_csv(f"{data_dir}/test_features.csv")
    sample_sub = pd.read_csv(f"{data_dir}/sample_submission.csv")
    return features, targets, test, sample_sub
```

```python
def clean_and_align(features, targets):
    """Drops nulls/duplicates and aligns rows by sig_id."""
    features = features.dropna(axis=0).drop_duplicates()
    targets = targets.dropna(axis=0).drop_duplicates()

    features = features.sort_values("sig_id").reset_index(drop=True)
    targets = targets.sort_values("sig_id").reset_index(drop=True)

    # Strict check on perfect alignment between the indipendent variables and targets
    assert features.shape[0] == targets.shape[0], "Row count mismatch between features and targets"
    assert (features["sig_id"].values == targets["sig_id"].values).all(), "Mismatched sig_id values"
    return features, targets
```

3. Feature engineering

Computation of aggregate statistics for each row and encoding of categorical variables.

```python
def add_row_statistics(df, g_cols, c_cols):
    """
    Sum, mean, standard deviation, kurtosis and skewness are extracted separately
    for the g- and c- feature groups to provide the model with summary information on
    general activation.
    """
    df = df.copy()
    df["g_sum"] = df[g_cols].sum(axis=1)
    df["g_mean"] = df[g_cols].mean(axis=1)
    df["g_std"] = df[g_cols].std(axis=1)
    df["g_kurtosis"] = df[g_cols].kurtosis(axis=1)
    df["g_skew"] = df[g_cols].skew(axis=1)

    df["c_sum"] = df[c_cols].sum(axis=1)
    df["c_mean"] = df[c_cols].mean(axis=1)
    df["c_std"] = df[c_cols].std(axis=1)
    df["c_kurtosis"] = df[c_cols].kurtosis(axis=1)
    df["c_skew"] = df[c_cols].skew(axis=1)
    return df


def encode_categoricals(df, dummy_cols, reference_columns=None):
    """One-hot encodes cp_time/cp_dose, aligned to the training columns."""
    df = pd.get_dummies(df, columns=dummy_cols, dtype=np.float64)
    if reference_columns is not None:
        df = df.reindex(columns=reference_columns, fill_value=0)
    return df
```

4. Model architectures

Definition of 3 neural networks in Pythorch:

- `MoAMLP (Multi-Layer Perceptron)`
- `TabularResNet`
- `FTTransformer`

```
class MoAMLP(nn.Module):
    name = "mlp"

    def __init__(self, input_dim, output_dim=206,
                 hidden_units=(1024, 1024, 512),
                 dropout_rates=(0.2, 0.3, 0.3)):
        super().__init__()
        self.batch_norm_input = nn.BatchNorm1d(input_dim)
        self.dropout_input = nn.Dropout(dropout_rates[0])

        self.dense1 = nn.Linear(input_dim, hidden_units[0])
        self.batch_norm1 = nn.BatchNorm1d(hidden_units[0])
        self.dropout1 = nn.Dropout(dropout_rates[1])

        self.dense2 = nn.Linear(hidden_units[0], hidden_units[1])
        self.batch_norm2 = nn.BatchNorm1d(hidden_units[1])
        self.dropout2 = nn.Dropout(dropout_rates[2])

        self.dense3 = nn.Linear(hidden_units[1], hidden_units[2])
        self.batch_norm3 = nn.BatchNorm1d(hidden_units[2])
        self.dropout3 = nn.Dropout(dropout_rates[2])

        self.output = nn.Linear(hidden_units[2], output_dim)

    def forward(self, x):
        x = self.dropout_input(self.batch_norm_input(x))

        x = F.silu(self.dense1(x))
        x = self.dropout1(self.batch_norm1(x))

        x = F.silu(self.dense2(x))
        x = self.dropout2(self.batch_norm2(x))

        x = F.silu(self.dense3(x))
        x = self.dropout3(self.batch_norm3(x))

        return self.output(x)
```

```
class TabularResNet(nn.Module):
    name = "resnet"

    def __init__(self, input_dim, output_dim=206, hidden_dim=512, dropout_rate=0.2):
        super().__init__()
        self.in_proj = nn.Sequential(
            nn.BatchNorm1d(input_dim),
            nn.Dropout(dropout_rate),
            nn.Linear(input_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.SiLU(),
        )
        self.block1 = self._make_block(hidden_dim, dropout_rate)
        self.block2 = self._make_block(hidden_dim, dropout_rate)
        self.out_proj = nn.Sequential(
            nn.SiLU(), nn.Dropout(dropout_rate), nn.Linear(hidden_dim, output_dim)
        )

    @staticmethod
    def _make_block(hidden_dim, dropout_rate):
        return nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.SiLU(),
            nn.Dropout(dropout_rate),
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
        )

    def forward(self, x):
        x = self.in_proj(x)
        x = F.silu(x + self.block1(x))
        x = F.silu(x + self.block2(x))
        return self.out_proj(x)


class FeatureTokenizer(nn.Module):
    """
    Projects each scalar numeric feature into a token vector of dimension
    d_token via an independent linear transform.
    """
```

```
    def __init__(self, num_features, d_token):
        super().__init__()
        self.weight = nn.Parameter(torch.empty(num_features, d_token))
        self.bias = nn.Parameter(torch.empty(num_features, d_token))
        nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5))
        nn.init.zeros_(self.bias)

    def forward(self, x):
        # x shape: (Batch, Features)
        # broadcast: (Batch, Features, 1) * (Features, d_token) -> (Batch, Features, d_token)
        return x.unsqueeze(-1) * self.weight + self.bias


class FTTransformer(nn.Module):
    """
    Feature Tokenizer + Transformer architecture. Turns the tabular input
    into a sequence of tokens (one per feature), prepends a global [CLS]
    token, and applies multi-head self-attention. Diversity relative to
    MLP/ResNet is guaranteed by the self-attention mechanism across features.
    """

    name = "ft_transformer"

    def __init__(self, input_dim, output_dim=206, d_token=64, n_blocks=2,
                 attention_heads=4, dropout=0.2):
        super().__init__()
        self.tokenizer = FeatureTokenizer(input_dim, d_token)

        # Global [CLS] token
        self.cls_token = nn.Parameter(torch.zeros(1, 1, d_token))
        nn.init.normal_(self.cls_token, std=0.02)

        # Transformer blocks (GELU is natively supported and is the standard choice)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_token,
            nhead=attention_heads,
            dim_feedforward=d_token * 4,
            dropout=dropout,
            activation="gelu",
            batch_first=True,
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=n_blocks)
```

```
        self.layer_norm = nn.LayerNorm(d_token)
        self.head = nn.Sequential(
            nn.Dropout(dropout),
            nn.Linear(d_token, output_dim)
        )

    def forward(self, x):
        # Tokenization: (B, input_dim) -> (B, input_dim, d_token)
        tokens = self.tokenizer(x)

        # Prepend [CLS] token: (B, 1 + input_dim, d_token)
        cls_tokens = self.cls_token.expand(x.size(0), -1, -1)
        x = torch.cat((cls_tokens, tokens), dim=1)

        # Transform via self-attention
        x = self.transformer(x)

        # Extract the output at the [CLS] token position (index 0)
        cls_out = self.layer_norm(x[:, 0, :])

        # Multi-label classification head
        return self.head(cls_out)
```

5. Evaluation metric

```
def compute_moa_log_loss(y_true, y_pred, eps=CLIP_EPS):
    """Mean binary log loss across all samples and all 206 targets, clipped
    to avoid log(0)."""
    y_pred = np.clip(y_pred, eps, 1 - eps)
    loss_matrix = -(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))
    return np.mean(loss_matrix)
```

6. Loss variants: focal loss and label smoothing

Definition of `FocalLossWithLogits` and `smooth_labels` function.

```
class FocalLossWithLogits(nn.Module):
    """Multi-label focal loss operating on raw logits (no sigmoid applied
    beforehand). Down-weights "easy" (already well-classified) samples,
    useful when positives are extremely rare, as in the MoA dataset."""

    def __init__(self, alpha=FOCAL_ALPHA, gamma=FOCAL_GAMMA):
        super().__init__()
```

```
        self.alpha = alpha
        self.gamma = gamma

    def forward(self, logits, targets):
        bce = F.binary_cross_entropy_with_logits(logits, targets, reduction="none")
        probs = torch.sigmoid(logits)
        pt = probs * targets + (1 - probs) * (1 - targets)
        focal_weight = (1 - pt) ** self.gamma
        alpha_weight = self.alpha * targets + (1 - self.alpha) * (1 - targets)
        return (alpha_weight * focal_weight * bce).mean()


def smooth_labels(y, smoothing):
    """y*(1-eps) + 0.5*eps: prevents the model from being overconfident
    (exact 0/1 targets), useful with strongly imbalanced classes."""
    if smoothing <= 0.0:
        return y
    return y * (1 - smoothing) + 0.5 * smoothing
```

7. Per-fold training routine

Training of one model on a specific fold and seed using:

- **AMP (Mixed Precision):** uses `torch.amp.GrandScaler` to speed up the training on GPUs maintaining stability.
- **Honest Validation:** validation computes the error avways on non-smoothed original targets, ensuring a reliable comparison regardless of the use of label smoothing during training.
- **In-RAM Checkpoint Saving:** the best model's `state_dict` is cloned to system RAM (CPU) and saved to disk only at the end of training, thereby avoiding numerous, slow I/O write cycles during every epoch where an improvement occurs.

```
def train_fold(fold, model, X_train, y_train, X_val, y_val,
               epochs=EPOCHS, batch_size=BATCH_SIZE, lr=LR,
               weight_decay=WEIGHT_DECAY, device=DEVICE, work_dir=WORK_DIR,
               seed=None, checkpoint_suffix="",
               early_stopping_patience=EARLY_STOPPING_PATIENCE,
               use_focal_loss=USE_FOCAL_LOSS, label_smoothing=LABEL_SMOOTHING,
               use_amp=USE_AMP, verbose=False):
    """Trains one model on one fold/seed with AMP, early stopping, optional
    focal loss and label smoothing. Returns the best validation predictions
    (probabilities) and the best validation loss.

    `verbose=False` (default) keeps this function silent so that running it
    45 times in parallel (3 models x 3 seeds x 5 folds) does not flood the
    notebook output with interleaved per-job lines; the calling code prints
    a single aggregated summary per model instead (see Step 9)."""
```

```
    if seed is not None:
        torch.manual_seed(seed)
        np.random.seed(seed)

    device = torch.device(device)
    use_amp = use_amp and device.type == "cuda"
    model = model.to(device)

    pin_memory = device.type == "cuda"
    train_loader = DataLoader(
        TensorDataset(torch.tensor(X_train, dtype=torch.float32),
                      torch.tensor(y_train, dtype=torch.float32)),
        batch_size=batch_size, shuffle=True, pin_memory=pin_memory,
    )
    val_loader = DataLoader(
        TensorDataset(torch.tensor(X_val, dtype=torch.float32),
                      torch.tensor(y_val, dtype=torch.float32)),
        batch_size=batch_size, shuffle=False, pin_memory=pin_memory,
    )

    criterion = FocalLossWithLogits() if use_focal_loss else nn.BCEWithLogitsLoss()
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min", factor=0.1, patience=3)
    try:
        scaler = torch.amp.GradScaler(device="cuda" if use_amp else "cpu", enabled=use_amp)
    except TypeError:
        scaler = torch.cuda.amp.GradScaler(enabled=use_amp)

    best_val_loss = float("inf")
    best_preds = None
    best_state_dict = None
    epochs_without_improvement = 0
    checkpoint_path = f"{work_dir}/best_{model.name}_fold_{fold}{checkpoint_suffix}.pth"

    for epoch in range(epochs):
        model.train()
        for x_batch, y_batch in train_loader:
            x_batch = x_batch.to(device, non_blocking=True)
            y_batch = y_batch.to(device, non_blocking=True)
            y_batch_train = smooth_labels(y_batch, label_smoothing)

            optimizer.zero_grad(set_to_none=True)
            with torch.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
```

```
            loss = criterion(model(x_batch), y_batch_train)
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()

    model.eval()
    val_loss = 0.0
    val_preds_list = []
    with torch.no_grad():
        for x_batch, y_batch in val_loader:
            x_batch = x_batch.to(device, non_blocking=True)
            y_batch = y_batch.to(device, non_blocking=True)
            with torch.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
                outputs = model(x_batch)
                batch_loss = criterion(outputs, y_batch)
            # Validation loss is ALWAYS computed on true (non-smoothed)
            # labels: an honest metric, independent of the label_smoothing setting.
            val_loss += batch_loss.item() * x_batch.size(0)
            val_preds_list.append(torch.sigmoid(outputs.float()).cpu().numpy())
    val_loss /= len(val_loader.dataset)
    scheduler.step(val_loss)

    improved = val_loss < best_val_loss
    if improved:
        best_val_loss = val_loss
        best_preds = np.vstack(val_preds_list)
        # The best checkpoint is kept in RAM (CPU) instead of being
        # written to disk on every improving epoch: the MLP typically
        # improves 15-20 times out of 35 epochs, so a write per epoch
        # would be wasted I/O. A single torch.save at the end gives an
        # identical final result.
        best_state_dict = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}
        epochs_without_improvement = 0
    else:
        epochs_without_improvement += 1

    if epochs_without_improvement >= early_stopping_patience:
        if verbose:
            print(f"--> Early stopping {model.name} fold {fold}{checkpoint_suffix} "
                  f"at epoch {epoch + 1}/{epochs} (no improvement for "
                  f"{early_stopping_patience} epochs)")
        break
```

```
    if verbose:
        print(f"--> Best val loss {model.name} fold {fold}{checkpoint_suffix}: {best_val_loss:.5f}")
    torch.save(best_state_dict, checkpoint_path)
    best_preds = np.clip(best_preds, CLIP_EPS, 1.0 - CLIP_EPS)

    # This process's CUDA allocator cache is NOT released
    # automatically when the function returns, only when the PROCESS ends.
    # With several loky workers sharing the same GPU (NN_JOBS_PER_GPU=2), a
    # cache left behind by one job is not visible to its sibling worker:
    # clearing it explicitly here returns it immediately to the GPU's free
    # pool for the next job.
    del model, optimizer, scaler, train_loader, val_loader
    if device.type == "cuda":
        torch.cuda.empty_cache()

    return best_preds, best_val_loss
```

8. Per-fold preprocessing pipelines

Definition of two types of data transformation: one with PCA (for MLP and ResNet) and the other without PCA (for FT-Transformer and CatBoost).

```
def fit_fold_pipeline(X_train, X_val, feature_cols, g_cols, c_cols, dummy_cols,
                      fold, work_dir=WORK_DIR, variance_threshold=VARIANCE_THRESHOLD):
    # Variance threshold, fit only on the fold's training split
    selector = VarianceThreshold(threshold=variance_threshold)
    X_train_vt = selector.fit_transform(X_train)
    X_val_vt = selector.transform(X_val)
    support = selector.get_support()
    selected_cols = [c for c, keep in zip(feature_cols, support) if keep]

    # Split continuous vs. dummy columns (dummy = cp_time_*, cp_dose_*)
    dummy_idx = [i for i, c in enumerate(selected_cols) if c in dummy_cols]
    cont_idx = [i for i, c in enumerate(selected_cols) if c not in dummy_cols]

    X_train_cont, X_val_cont = X_train_vt[:, cont_idx], X_val_vt[:, cont_idx]
    X_train_dummy, X_val_dummy = X_train_vt[:, dummy_idx], X_val_vt[:, dummy_idx]
    cont_cols = [selected_cols[i] for i in cont_idx]

    g_idx_in_cont = [i for i, c in enumerate(cont_cols) if c in g_cols]
    c_idx_in_cont = [i for i, c in enumerate(cont_cols) if c in c_cols]

    # RankGauss only on the continuous columns (not on the 0/1 dummies)
    qt = QuantileTransformer(n_quantiles=N_QUANTILES, output_distribution="normal",
                             random_state=RANDOM_STATE)
```

```
    X_train_tf = qt.fit_transform(X_train_cont)
    X_val_tf = qt.transform(X_val_cont)

    # PCA on the g- and c- blocks
    pca_g = PCA(n_components=PCA_G_COMPONENTS, random_state=RANDOM_STATE)
    train_pca_g = pca_g.fit_transform(X_train_tf[:, g_idx_in_cont])
    val_pca_g = pca_g.transform(X_val_tf[:, g_idx_in_cont])

    pca_c = PCA(n_components=PCA_C_COMPONENTS, random_state=RANDOM_STATE)
    train_pca_c = pca_c.fit_transform(X_train_tf[:, c_idx_in_cont])
    val_pca_c = pca_c.transform(X_val_tf[:, c_idx_in_cont])

    # Final concatenation: transformed continuous + PCA + raw dummies
    X_train_final = np.hstack([X_train_tf, train_pca_g, train_pca_c, X_train_dummy])
    X_val_final = np.hstack([X_val_tf, val_pca_g, val_pca_c, X_val_dummy])

    # Persist the whole fold pipeline in a single file
    joblib.dump(
        {"selector": selector, "qt": qt, "pca_g": pca_g, "pca_c": pca_c,
         "cont_idx": cont_idx, "dummy_idx": dummy_idx,
         "g_idx_in_cont": g_idx_in_cont, "c_idx_in_cont": c_idx_in_cont},
        f"{work_dir}/fold_pipeline_{fold}.joblib",
    )
    return X_train_final, X_val_final


def apply_fold_pipeline(X, fold, work_dir=WORK_DIR):
    """Applies the pipeline already fitted for that fold (training or
    inference alike: a pure transform, no fitting)."""
    artifacts = joblib.load(f"{work_dir}/fold_pipeline_{fold}.joblib")
    X_vt = artifacts["selector"].transform(X)
    X_cont = X_vt[:, artifacts["cont_idx"]]
    X_dummy = X_vt[:, artifacts["dummy_idx"]]
    X_tf = artifacts["qt"].transform(X_cont)
    pca_g_out = artifacts["pca_g"].transform(X_tf[:, artifacts["g_idx_in_cont"]])
    pca_c_out = artifacts["pca_c"].transform(X_tf[:, artifacts["c_idx_in_cont"]])
    return np.hstack([X_tf, pca_g_out, pca_c_out, X_dummy])
```

```
def fit_fold_pipeline_raw(X_train, X_val, feature_cols, dummy_cols,
                          fold, work_dir=WORK_DIR, variance_threshold=VARIANCE_THRESHOLD,
                          reuse_if_cached=True):
    """"Raw" pipeline for tree-based models (and for the FT-Transformer,
    which reuses it to get one token per original feature, without PCA):
```

```
    VarianceThreshold + RankGauss on the continuous columns, NO PCA.

    PCA rotates the axes towards directions of maximum global variance:
    this helps dense networks (which combine all inputs in every neuron)
    but strips features of their original per-feature meaning, which is
    exactly what a tree split relies on (a split on a PCA component is an
    oblique hyperplane in the original space, almost never aligned with a
    single gene/cell-viability column -> uninformative splits: LightGBM on
    PCA features gave an OOF log loss of 0.045 versus 0.016 for the dense
    nets). The dummy columns (cp_time/cp_dose one-hot) are left raw, as in
    the main pipeline.

    OPTIMISATION: if this pipeline has already been fitted for this fold
    (e.g. during the neural-network cross-validation, for the
    FT-Transformer) the .joblib file already exists on disk: it is reused
    in transform-only mode instead of refitting VarianceThreshold /
    QuantileTransformer a second time on the same data (the fit is
    deterministic for a given fold/random_state, so the result is
    identical either way)."""
    cache_path = f"{work_dir}/fold_pipeline_raw_{fold}.joblib"
    if reuse_if_cached and os.path.exists(cache_path):
        return (apply_fold_pipeline_raw(X_train, fold, work_dir=work_dir),
                apply_fold_pipeline_raw(X_val, fold, work_dir=work_dir))

    selector = VarianceThreshold(threshold=variance_threshold)
    X_train_vt = selector.fit_transform(X_train)
    X_val_vt = selector.transform(X_val)
    support = selector.get_support()
    selected_cols = [c for c, keep in zip(feature_cols, support) if keep]

    dummy_idx = [i for i, c in enumerate(selected_cols) if c in dummy_cols]
    cont_idx = [i for i, c in enumerate(selected_cols) if c not in dummy_cols]

    X_train_cont, X_val_cont = X_train_vt[:, cont_idx], X_val_vt[:, cont_idx]
    X_train_dummy, X_val_dummy = X_train_vt[:, dummy_idx], X_val_vt[:, dummy_idx]

    qt = QuantileTransformer(n_quantiles=N_QUANTILES, output_distribution="normal",
                             random_state=RANDOM_STATE)
    X_train_tf = qt.fit_transform(X_train_cont)
    X_val_tf = qt.transform(X_val_cont)

    joblib.dump(
        {"selector": selector, "qt": qt, "cont_idx": cont_idx, "dummy_idx": dummy_idx},
```

```
        f"{work_dir}/fold_pipeline_raw_{fold}.joblib",
    )

    X_train_final = np.hstack([X_train_tf, X_train_dummy])
    X_val_final = np.hstack([X_val_tf, X_val_dummy])
    return X_train_final, X_val_final


def apply_fold_pipeline_raw(X, fold, work_dir=WORK_DIR):
    artifacts = joblib.load(f"{work_dir}/fold_pipeline_raw_{fold}.joblib")
    X_vt = artifacts["selector"].transform(X)
    X_cont = X_vt[:, artifacts["cont_idx"]]
    X_dummy = X_vt[:, artifacts["dummy_idx"]]
    X_tf = artifacts["qt"].transform(X_cont)
    return np.hstack([X_tf, X_dummy])
```

9. Cross-validation: neural network ensemble (MLP, ResNet, FT-Transformer)

GPU multiprocessing orchestration via `joblib` (`loky` library) to train 45 parallel jobs (3 models × 3 seeds × 5 folds).

```
MODEL_CLASSES = {"mlp": MoAMLP, "resnet": TabularResNet, "ft_transformer": FTTransformer}
MODEL_NAMES = tuple(MODEL_CLASSES.keys())


def _build_model(model_name, input_dim, output_dim):
    return MODEL_CLASSES[model_name](input_dim=input_dim, output_dim=output_dim)


def _resolve_device(gpu_id):
    """Single place for the GPU-assignment logic: jobs are distributed
    round-robin across all available GPUs (rather than converging on a
    single one), so both T4s are used evenly."""
    if gpu_id is not None and N_GPUS > 0:
        torch.cuda.set_device(gpu_id)
        return torch.device(f"cuda:{gpu_id}")
    return torch.device("cpu")


def _train_nn_job(fold, model_name, seed, X_train, y_train, X_val, y_val, gpu_id,
                  batch_size=None):
    """
    Runs in a worker process (joblib/loky). Receives features that have
    already been preprocessed with the pipeline appropriate for this model:
    PCA features for mlp/resnet, raw RankGauss features (no PCA, one token
```

```
    per original feature) for the FT-Transformer.
    """
    device = _resolve_device(gpu_id)

    input_dim = X_train.shape[1]
    model = _build_model(model_name, input_dim=input_dim, output_dim=y_train.shape[1])

    bs = batch_size or BATCH_SIZE_BY_MODEL.get(model_name, BATCH_SIZE)
    preds, val_loss = train_fold(
        fold, model, X_train, y_train, X_val, y_val,
        device=device, seed=seed, batch_size=bs,
        checkpoint_suffix=f"_seed{seed}",
    )
    return model_name, fold, seed, preds, val_loss


def run_cross_validation(X, Y, feature_cols, g_cols, c_cols, dummy_cols,
                         n_folds=N_FOLDS, seeds=SEEDS, n_jobs=N_JOBS_NN,
                         model_names=MODEL_NAMES):
    mskf = MultilabelStratifiedKFold(n_splits=n_folds, shuffle=True, random_state=RANDOM_STATE)
    splits = list(mskf.split(X, Y))

    oof = {name: np.zeros_like(Y, dtype=np.float64) for name in model_names}

    # Per-fold preprocessing, run sequentially (CPU-bound and already
    # fast: fitting 5 pipelines does not benefit from multi-GPU
    # parallelism). Two variants:
    #   - "pca": RankGauss + PCA (mlp, resnet)         -> fit_fold_pipeline
    #   - "raw": RankGauss WITHOUT PCA (ft_transformer) -> fit_fold_pipeline_raw
    # "raw" is the SAME pipeline reused later by CatBoost: fitting it
    # here does not refit it twice, because fit_fold_pipeline_raw
    # detects the cache on disk and only transforms in that case.
    fold_data = {}
    for fold, (train_idx, val_idx) in enumerate(splits):
        print(f"Preprocessing fold {fold}...")
        X_train, y_train = X[train_idx], Y[train_idx]
        X_val, y_val = X[val_idx], Y[val_idx]

        X_train_pca, X_val_pca = fit_fold_pipeline(
            X_train, X_val, feature_cols, g_cols, c_cols, dummy_cols, fold
        )
        X_train_raw, X_val_raw = fit_fold_pipeline_raw(
            X_train, X_val, feature_cols, dummy_cols, fold
```

```
    )
    fold_data[fold] = {
        "pca": (X_train_pca, X_val_pca),
        "raw": (X_train_raw, X_val_raw),
        "y_train": y_train, "y_val": y_val, "val_idx": val_idx,
    }

# One job per (fold, model, seed) = 45 jobs with 3 models x 3 seeds x
# 5 folds, all submitted to a SINGLE pool assigned round-robin across
# the available GPUs: both T4s stay occupied for the whole CV. Each
# job explicitly receives its own fold's already-preprocessed
# features (see `_job_arrays`), so there is no risk of a job
# accidentally training on the wrong fold's split.
jobs = [(fold, model_name, seed)
        for fold in range(n_folds)
        for model_name in model_names
        for seed in seeds]

def _job_arrays(fold, model_name):
    key = "raw" if model_name == "ft_transformer" else "pca"
    X_train_final, X_val_final = fold_data[fold][key]
    return X_train_final, fold_data[fold]["y_train"], X_val_final, fold_data[fold]["y_val"]

print(f"Launching {len(jobs)} training jobs "
      f"({len(model_names)} models x {len(seeds)} seeds x {n_folds} folds) "
      f"across {n_jobs} parallel workers...")

results = Parallel(n_jobs=n_jobs, backend="loky")(
    delayed(_train_nn_job)(
        fold, model_name, seed, *_job_arrays(fold, model_name),
        gpu_id=(i % N_GPUS) if N_GPUS > 0 else None,
    )
    for i, (fold, model_name, seed) in enumerate(jobs)
)

# Explicitly shuts down the loky worker pool. torch does not
# release a process's CUDA allocator cache when a job finishes, only
# when the PROCESS itself ends: without this shutdown, memory would
# stay occupied by the (now idle) workers while predict_test_set_nn and
# then CatBoost start up next, increasing the risk of OOM in the
# following stages.
get_reusable_executor().shutdown(wait=True)
```

```
    # Aggregate: average over seeds for each (fold, model) pair, and
    # print one concise summary line per (fold, model) instead of the
    # 45 raw per-job lines.
    seed_preds = {(fold, name): [] for fold in range(n_folds) for name in model_names}
    seed_losses = {(fold, name): [] for fold in range(n_folds) for name in model_names}
    for model_name, fold, seed, preds, val_loss in results:
        seed_preds[(fold, model_name)].append(preds)
        seed_losses[(fold, model_name)].append(val_loss)

    for fold in range(n_folds):
        val_idx = fold_data[fold]["val_idx"]
        for name in model_names:
            oof[name][val_idx] = np.mean(seed_preds[(fold, name)], axis=0)
            mean_loss = np.mean(seed_losses[(fold, name)])
            print(f"  fold {fold} | {name:14s} | mean val loss over {len(seeds)} seeds: {mean_loss:.5f}")

    return oof, splits
```

```
def _predict_nn_job(fold, model_name, seed, X_test_final, num_targets, gpu_id,
                    work_dir=WORK_DIR, batch_size=None):
    device = _resolve_device(gpu_id)

    input_dim = X_test_final.shape[1]
    model = _build_model(model_name, input_dim, num_targets)
    ckpt = f"{work_dir}/best_{model.name}_fold_{fold}_seed{seed}.pth"
    model.load_state_dict(torch.load(ckpt, map_location=device, weights_only=True))
    model = model.to(device).eval()

    # Inference is run in mini-batches rather than a single monolithic
    # forward pass over the whole test set: for the FT-Transformer, one
    # attention matrix of shape (n_test_rows x heads x seq_len x seq_len)
    # with seq_len ~870 and thousands of test rows would require tens of
    # GB in a single pass, far beyond a T4's VRAM. mlp/resnet are safer in
    # this respect but still benefit from batching on large test sets.
    bs = batch_size or INFERENCE_BATCH_SIZE_BY_MODEL.get(model_name, BATCH_SIZE)
    test_loader = DataLoader(
        TensorDataset(torch.tensor(X_test_final, dtype=torch.float32)),
        batch_size=bs, shuffle=False,
    )
    preds_chunks = []
    with torch.no_grad():
        for (x_batch,) in test_loader:
            x_batch = x_batch.to(device, non_blocking=True)
```

```
            preds_chunks.append(torch.sigmoid(model(x_batch)).cpu().numpy())
    preds = np.vstack(preds_chunks)

    del model
    if device.type == "cuda":
        torch.cuda.empty_cache()
    return model_name, preds


def predict_test_set_nn(X_test_final_by_fold_pca, X_test_final_by_fold_raw, num_targets,
                        n_folds=N_FOLDS, seeds=SEEDS, n_jobs=N_JOBS_NN, model_names=MODEL_NAMES):
    """
    Averages test-set predictions across all folds and seeds for each
    neural-network model. Each model receives the test features from the
    preprocessing variant it was trained on: PCA features for mlp/resnet,
    raw RankGauss features (no PCA) for the FT-Transformer.
    """
    n_samples = next(iter(X_test_final_by_fold_pca.values())).shape[0]
    preds = {name: np.zeros((n_samples, num_targets)) for name in model_names}
    n_models = n_folds * len(seeds)

    jobs = [(fold, model_name, seed)
            for fold in range(n_folds)
            for model_name in model_names
            for seed in seeds]

    def _test_features(fold, model_name):
        return (X_test_final_by_fold_raw[fold] if model_name == "ft_transformer"
                else X_test_final_by_fold_pca[fold])

    results = Parallel(n_jobs=n_jobs, backend="loky")(
        delayed(_predict_nn_job)(
            fold, model_name, seed, _test_features(fold, model_name), num_targets,
            gpu_id=(i % N_GPUS) if N_GPUS > 0 else None,
        )
        for i, (fold, model_name, seed) in enumerate(jobs)
    )
    get_reusable_executor().shutdown(wait=True)

    for model_name, p in results:
        preds[model_name] += p / n_models

    return preds
```

10. Third model: CatBoost on raw (non-PCA) features

CatBoost model training optimizing all targets at once with `MultiLogLoss`.

```python
def train_catboost_fold(fold, X_train, y_train, X_val, y_val, work_dir=WORK_DIR,
                        iterations=N_ESTIMATORS_CATBOOST, learning_rate=CATBOOST_LEARNING_RATE,
                        depth=CATBOOST_DEPTH, gpu_id=0):
    from catboost import CatBoostClassifier

    num_targets = y_train.shape[1]
    # Some of the 206 MoA targets have very few positives: in certain folds
    # a target's training column can be ALL zero (no positives in that
    # fold). CatBoost's MultiLogloss requires both classes to be present
    # for EVERY target passed to fit(), even without an eval_set
    # ("All train targets are equal"). This mirrors the equivalent
    # per-target check used in the original single-target boosting
    # baseline (`if len(np.unique(y_train_col)) < 2`), applied here to the
    # whole multi-output matrix at once: only non-constant targets are
    # included in the fit; for the others, the constant value observed in
    # the training fold (almost always 0.0) is used directly.
    col_is_constant = np.array([len(np.unique(y_train[:, t])) < 2 for t in range(num_targets)])
    active_cols = np.where(~col_is_constant)[0]
    constant_cols = np.where(col_is_constant)[0]
    constant_vals = y_train.mean(axis=0)  # only valid at the constant_cols positions

    if len(constant_cols) > 0:
        print(f"[catboost | fold {fold}] {len(constant_cols)} constant target(s) in "
              f"the training fold (excluded from the fit, using the observed constant)")

    model = CatBoostClassifier(
        loss_function="MultiLogloss",   # a single fit over the active targets,
        iterations=iterations,          # rather than N separate binary fits
        learning_rate=learning_rate,
        depth=depth,
        task_type="GPU" if N_GPUS > 0 else "CPU",
        devices=str(gpu_id) if N_GPUS > 0 else None,
        random_seed=RANDOM_STATE,
        verbose=False,
    )
    # No eval_set / early_stopping_rounds: fixed number of iterations.
    model.fit(X_train, y_train[:, active_cols])

    val_preds = np.zeros((X_val.shape[0], num_targets))
    if len(active_cols) > 0:
        val_preds[:, active_cols] = model.predict_proba(X_val)
```

```
    if len(constant_cols) > 0:
        val_preds[:, constant_cols] = constant_vals[constant_cols]
    val_preds = np.clip(val_preds, CLIP_EPS, 1 - CLIP_EPS)
    val_loss = compute_moa_log_loss(y_val, val_preds)

    model.save_model(f"{work_dir}/catboost_fold_{fold}.cbm")
    joblib.dump(
        {"active_cols": active_cols, "constant_cols": constant_cols, "constant_vals": constant_vals},
        f"{work_dir}/catboost_fold_{fold}_meta.joblib",
    )
    print(f"[catboost | fold {fold}] val_loss={val_loss:.5f}")
    return val_preds, val_loss


def run_cross_validation_catboost(X, Y, feature_cols, dummy_cols, splits, work_dir=WORK_DIR):
    """Reuses the SAME splits as run_cross_validation (same random_state
    passed to MultilabelStratifiedKFold) but a different preprocessing
    pipeline: RankGauss without PCA (fit_fold_pipeline_raw). Runs
    sequentially per fold rather than through joblib multiprocessing like
    the neural nets: a multi-output CatBoost fit on GPU is already fast,
    and staying single-process avoids GPU contention."""
    oof_catboost = np.zeros_like(Y, dtype=np.float64)

    for fold, (train_idx, val_idx) in enumerate(splits):
        X_train_raw, y_train = X[train_idx], Y[train_idx]
        X_val_raw, y_val = X[val_idx], Y[val_idx]

        X_train_final, X_val_final = fit_fold_pipeline_raw(
            X_train_raw, X_val_raw, feature_cols, dummy_cols, fold, work_dir=work_dir
        )
        val_preds, _ = train_catboost_fold(
            fold, X_train_final, y_train, X_val_final, y_val,
            gpu_id=(fold % N_GPUS) if N_GPUS > 0 else 0, work_dir=work_dir,
        )
        oof_catboost[val_idx] = val_preds

    return oof_catboost


def predict_test_set_catboost(X_test_raw, feature_cols, dummy_cols, num_targets,
                              n_folds=N_FOLDS, work_dir=WORK_DIR):
    from catboost import CatBoostClassifier
```

```
    n_samples = X_test_raw.shape[0]
    preds = np.zeros((n_samples, num_targets))
    for fold in range(n_folds):
        X_test_final = apply_fold_pipeline_raw(X_test_raw, fold, work_dir=work_dir)
        meta = joblib.load(f"{work_dir}/catboost_fold_{fold}_meta.joblib")
        active_cols, constant_cols, constant_vals = (
            meta["active_cols"], meta["constant_cols"], meta["constant_vals"]
        )

        model = CatBoostClassifier()
        model.load_model(f"{work_dir}/catboost_fold_{fold}.cbm")

        fold_preds = np.zeros((n_samples, num_targets))
        if len(active_cols) > 0:
            fold_preds[:, active_cols] = model.predict_proba(X_test_final)
        if len(constant_cols) > 0:
            fold_preds[:, constant_cols] = constant_vals[constant_cols]

        preds += np.clip(fold_preds, CLIP_EPS, 1 - CLIP_EPS) / n_folds
    return preds
```

11. Ensemble blending

Computation of the ideal weights for combining the four models and evaluate performance using a nested approach.

```
def optimize_blend_weights(oof_list, Y):
    """
    Optimises the blend weights on the simplex (weights >= 0, sum = 1)
    via a softmax reparameterisation: free logits are optimised (no
    explicit constraint), then softmax is applied to obtain the weights.
    Nelder-Mead is used instead of SLSQP with an explicit equality
    constraint: on an objective averaged over millions of elements
    (N x 206 targets), SLSQP's finite-difference gradient can appear
    numerically zero at iteration 0, causing the optimiser to terminate
    WITHOUT moving from x0 and WITHOUT raising an error (observed result:
    weights all exactly equal to 1/n_models). Nelder-Mead is derivative-free
    and has no such fragility; it is robust for a low-dimensional problem
    like this one (a handful of models).
    """
    n_models = len(oof_list)

    def objective(logits):
        w = np.exp(logits - logits.max())
        w = w / w.sum()
```

```
        blend = sum(wi * o for wi, o in zip(w, oof_list))
        return compute_moa_log_loss(Y, blend)

    x0 = np.zeros(n_models)  # uniform logits -> softmax(x0) = 1/n_models
    result = minimize(objective, x0=x0, method="Nelder-Mead",
                      options={"xatol": 1e-6, "fatol": 1e-9, "maxiter": 3000})

    if not result.success:
        print(f"[blend] WARNING: optimisation did not converge cleanly "
              f"({result.message}); using the best point found regardless.")

    w = np.exp(result.x - result.x.max())
    w = w / w.sum()
    return w


def nested_blend_validation(oof_list, Y, splits):
    """
    Honest estimate of the ensemble's performance: for each evaluation
    fold, the blend weights are optimised ONLY on the other folds, never on
    the same data the score is measured on. This avoids overestimating the
    blend's gain.
    """
    fold_scores, fold_weights = [], []

    for val_fold_idx, (_, val_idx) in enumerate(splits):
        train_mask = np.ones(Y.shape[0], dtype=bool)
        train_mask[val_idx] = False

        oof_train = [oof[train_mask] for oof in oof_list]
        oof_val = [oof[val_idx] for oof in oof_list]

        weights = optimize_blend_weights(oof_train, Y[train_mask])
        blend_val = sum(w * o for w, o in zip(weights, oof_val))
        score = compute_moa_log_loss(Y[val_idx], blend_val)

        fold_scores.append(score)
        fold_weights.append(weights)
        print(f"[nested blend] fold {val_fold_idx}: score={score:.5f} weights={np.round(weights, 3)}")

    mean_score, std_score = float(np.mean(fold_scores)), float(np.std(fold_scores))
    final_weights = np.mean(fold_weights, axis=0)
    final_weights = final_weights / final_weights.sum()
```

```
    print(f"Honest nested-blend score: {mean_score:.5f} +/- {std_score:.5f}")
    print(f"Final weights (nested mean): {np.round(final_weights, 4)}")
    return final_weights, mean_score, std_score
```

12. Experiment logging

Definition of a function that saves all of the hyperparameters and the experiment metrics in JSON format, adding also a timestamp.

```
def log_experiment(run_name, params, metrics, log_dir=LOG_DIR):
    os.makedirs(log_dir, exist_ok=True)
    record = {
        "run_name": run_name,
        "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
        "params": params,
        "metrics": metrics,
    }
    path = os.path.join(log_dir, f"{run_name}_{int(time.time())}.json")
    with open(path, "w") as f:
        json.dump(record, f, indent=2)
    print(f"Experiment log saved: {path}")
```

13. Submission building and sanity checks

`submission.csv` creation and lanch of the entire end-to-end pipeline. Control rows (`cp_type == "ctl_vehicle"`) are excluded during the experiment as per the competition rules, because they are never going to present any mechanismo of action.

```
def build_submission(raw_test, sample_submission, target_cols, trt_preds, trt_mask):
    """
    Explicitly overwrites the predicted values for the test dataset with 0.0
    if they correspond to control vehicles, thereby mathematically improving the score.
    """
    n_samples = len(raw_test)
    num_targets = len(target_cols)

    full_preds = np.zeros((n_samples, num_targets), dtype=np.float64)
    trt_indices = raw_test[trt_mask].index.values
    full_preds[trt_indices] = trt_preds
    full_preds = np.clip(full_preds, CLIP_EPS, 1.0 - CLIP_EPS)

    submission = sample_submission.copy()
    submission[target_cols] = full_preds
    submission.loc[raw_test["cp_type"] == CTL_LABEL, target_cols] = 0.0
    return submission
```

```python
def sanity_check_submission(submission, sample_submission, raw_test, target_cols):
    """
    Validates the shape and checks for the absence of NaNs and infinite values,
    ensuring that the maximum check value is exactly 0.0 before export,
    thereby preventing submission failures on Kaggle.
    """
    assert submission.shape == sample_submission.shape, f"Wrong shape: {submission.shape}"
    assert list(submission.columns) == list(sample_submission.columns), "Column mismatch"
    assert submission.isna().sum().sum() == 0, "NaNs present"
    assert np.isinf(submission[target_cols].values).sum() == 0, "Infinite values present"

    ctl_ids = raw_test.loc[raw_test["cp_type"] == CTL_LABEL, "sig_id"]
    ctl_preds = submission[submission["sig_id"].isin(ctl_ids)][target_cols].values
    assert ctl_preds.max() == 0.0, f"ctl_vehicle rows not zeroed: max={ctl_preds.max()}"

    trt_preds = submission[~submission["sig_id"].isin(ctl_ids)][target_cols].values
    print(f"Min trt_cp probability: {trt_preds.min():.2e} | Max: {trt_preds.max():.2e} | Mean: {trt_preds.mean():.5f}")
    print("Sanity check passed.")
```

14. Main pipeline

Execution of all the previous steps, in order to generate the `submission.csv` file.

```python
raw_features, raw_targets, raw_test, sample_submission = load_raw_data(DATA_DIR)
features, targets = clean_and_align(raw_features, raw_targets)

trt_mask_train = features["cp_type"] != CTL_LABEL
features = features[trt_mask_train].reset_index(drop=True)
targets = targets[trt_mask_train].reset_index(drop=True)

g_cols = [c for c in features.columns if c.startswith("g-")]
c_cols = [c for c in features.columns if c.startswith("c-")]

features = add_row_statistics(features, g_cols, c_cols)
features = encode_categoricals(features, dummy_cols=["cp_time", "cp_dose"])

feature_cols = [c for c in features.columns if c not in ("sig_id", "cp_type")]
target_cols = [c for c in targets.columns if c != "sig_id"]
dummy_cols = [c for c in feature_cols if c.startswith("cp_time_") or c.startswith("cp_dose_")]

X = features[feature_cols].values
Y = targets[target_cols].values
```

```
# MLP + ResNet + FT-Transformer, multi-seed
t0 = time.time()
oof, splits = run_cross_validation(X, Y, feature_cols, g_cols, c_cols, dummy_cols)
t1 = time.time()
for name in MODEL_NAMES:
    print(f"{name:14s} log loss OOF: {compute_moa_log_loss(Y, oof[name]):.5f}")

# CatBoost, RankGauss features WITHOUT PCA
oof_catboost = run_cross_validation_catboost(X, Y, feature_cols, dummy_cols, splits)
t2 = time.time()
print(f"catboost       log loss OOF: {compute_moa_log_loss(Y, oof_catboost):.5f}")
print(f"[timing] NN CV: {(t1 - t0) / 60:.1f} min | CatBoost CV: {(t2 - t1) / 60:.1f} min")

ALL_MODEL_NAMES = MODEL_NAMES + ("catboost",)
oof["catboost"] = oof_catboost

# Nested CV for the blend weights
oof_list = [oof[name] for name in ALL_MODEL_NAMES]
final_weights, nested_mean, nested_std = nested_blend_validation(oof_list, Y, splits)
weights = dict(zip(ALL_MODEL_NAMES, final_weights))

oof_blend = sum(weights[name] * oof[name] for name in ALL_MODEL_NAMES)
blend_score_naive = compute_moa_log_loss(Y, oof_blend)
print(f"Ensemble log loss OOF (same data used to fit the weights): {blend_score_naive:.5f}")
print(f"Ensemble log loss, nested (honest): {nested_mean:.5f} +/- {nested_std:.5f}")

log_experiment(
    run_name=f"moa_ensemble_",
    params={"seeds": SEEDS, "variance_threshold": VARIANCE_THRESHOLD,
            "use_focal_loss": USE_FOCAL_LOSS, "label_smoothing": LABEL_SMOOTHING,
            "early_stopping_patience": EARLY_STOPPING_PATIENCE,
            "model_names": list(ALL_MODEL_NAMES),
            "catboost_iterations": N_ESTIMATORS_CATBOOST,
            "catboost_depth": CATBOOST_DEPTH},
    metrics={
        **{f"{name}_oof": compute_moa_log_loss(Y, oof[name]) for name in ALL_MODEL_NAMES},
        "blend_oof_naive": blend_score_naive,
        "blend_oof_nested": nested_mean,
        **{f"w_{name}": float(weights[name]) for name in ALL_MODEL_NAMES},
    },
)

# Test set: preprocessing and inference for all models
```

```
trt_mask_test = raw_test["cp_type"] != CTL_LABEL
test_trt = raw_test[trt_mask_test].reset_index(drop=True)
test_trt = add_row_statistics(test_trt, g_cols, c_cols)
test_trt = encode_categoricals(test_trt, dummy_cols=["cp_time", "cp_dose"],
                               reference_columns=feature_cols)
X_test_raw = test_trt[feature_cols].values

# Two consistent feature variants, matching what each model was trained on
# (see step 9): PCA features for mlp/resnet, raw RankGauss features (no
# PCA, the same pipeline already reused by CatBoost) for the FT-Transformer.
X_test_final_by_fold_pca = {
    fold: apply_fold_pipeline(X_test_raw, fold) for fold in range(N_FOLDS)
}
X_test_final_by_fold_raw = {
    fold: apply_fold_pipeline_raw(X_test_raw, fold) for fold in range(N_FOLDS)
}

t3 = time.time()
preds_test = predict_test_set_nn(
    X_test_final_by_fold_pca, X_test_final_by_fold_raw, num_targets=Y.shape[1]
)
preds_test["catboost"] = predict_test_set_catboost(
    X_test_raw, feature_cols, dummy_cols, num_targets=Y.shape[1]
)
final_test_preds = sum(weights[name] * preds_test[name] for name in ALL_MODEL_NAMES)
t4 = time.time()
print(f"[timing] Test-set inference: {(t4 - t3) / 60:.1f} min | Total run: {(t4 - t0) / 60:.1f} min")

submission = build_submission(raw_test, sample_submission, target_cols,
                              final_test_preds, trt_mask_test)

# Final sanity check, run on the object actually written to disk, not on an
# intermediate version.
sanity_check_submission(submission, sample_submission, raw_test, target_cols)

submission.to_csv("submission.csv", index=False)
print("submission.csv written successfully.")
```